

Document number	P2623R2
Date	2022-09-12
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

implicit constant initialization

Table of contents

- implicit constant initialization
 - Changelog
 - Abstract
 - Motivating examples
 - Classes not Having Value Semantics
 - Returned References to Temporaries
 - Proposed Wording
 - In Depth Rationale
 - Why not before
 - Storage Duration
 - Constant Expressions
 - Constant Initialization
 - constant definition in class definitions
 - constant definition in function body
 - constant definition in function parameter and arguments
 - Impact on current proposals
 - p2255r2
 - p2576r0
 - Past
 - n1511
 - n2235
 - Present
 - C Standard Compound Literals

- C++ Standard
- Outstanding Issues
 - P1018R16
 - LWG2432 `initializer_list` assignability
 - CWG900 Lifetime of temporaries in range-based for
 - CWG1864 List-initialization of array objects
 - CWG2111 Array temporaries in reference binding
 - CWG914 Value-initialization of array types
- Other Anonymous Things
- Other languages
- Summary
- Frequently Asked Questions
- References

Changelog

R2

- added new “Other Anonymous Things” section which covers lambda functions and coroutines
- elaborated on the “Summary” section
- added to “Frequently Asked Questions” section information concerning breakage, use, impact on static analyzers, `constexpr`, implementability and teachability
- verbiage clarifications and pruning
- added the fundamental flaw segment

R1

- Clarified existing attribution by
 - adding more links to existing references
 - judiciously used boxes to group quoted material together to improve visibility
- Numerous verbiage clarifications
- Greatly reduced the `why not before` section
- `std::initializer_list` example automatically fixed without using `const`

Abstract

“Lifetime issues with references to temporaries can lead to fatal and subtle runtime errors. This applies to both.” ^[1]

- *“Returned references (for example, when using strings or maps) and”* ^[1:1]
- *“Returned objects that do not have value semantics (for example using `std::string_view`).”* ^[1:2]

This paper proposes the standard adopt existing common practices in order to eliminate dangling in some cases and in many other cases, greatly reduce them.

Motivating Examples

“Let’s motivate the feature for both, classes not having value semantics and references.” ^[1:3]

“Classes not Having Value Semantics” ^[1:4]

“C++ allows the definition of classes that do not have value semantics. One famous example is `std::string_view`: The lifetime of a `string_view` object is bound to an underlying string or character sequence.” ^[1:5]

“Because `string` has an implicit conversion to `string_view`, it is easy to accidentally program a `string_view` to a character sequence that doesn’t exist anymore.” ^[1:6]

“A trivial example is this:” ^[1:7]

```
std::string_view sv = "hello world"s; // immediate dangling reference
```

It is clear from this `string_view` example that it dangles because `sv` is a reference and `"hello world"s` is a temporary. ***What is being proposed is that same example doesn’t dangle!*** If the evaluated constant expression `"hello world"s` had static storage duration just like the string literal `"hello world"` has static storage duration ^[2] (5.13.5 String literals [lex.string]) then `sv` would be a reference to something that is global and as such would not dangle. This is reasonable based on how programmers reason about constants being immutable variables and temporaries which are known at compile time and do not change for the life of the program. There are a few facts to take note of in the previous example.

- constants, whether `"hello world"` or `"hello world"s`, **are not expected by programmers** to dangle but rather to be immutable and available for the life of the program in other words `const` and static storage duration

- the constexpr constructor of std::string_view **expects** a const argument
- sv is constant-initialized [2:1] (7.7 Constant expressions [expr.const])

Dangling “can occur more indirectly as follows:” [1:8]

```
std::string operator+ (std::string_view s1, std::string_view s2) {
    return std::string{s1} + std::string{s2};
}
std::string_view sv = "hi";
sv = sv + sv; // fatal runtime error: sv refers to deleted temporary string
```

The problem here is that the lifetime of the temporary is bound to the statement in which it was created, instead of the block that contains said expression.

Working Draft, Standard for Programming Language C++ [2:2]

“6.7.7 Temporary objects”

“Temporary objects are destroyed as the last step in evaluating the full-expression (6.9.1) that (lexically) contains the point where they were created. This is true even if that evaluation ends in throwing an exception. The value computations and side effects of destroying a temporary object are associated only with the full-expression, not with any specific subexpression.”

Had the temporary been bound to the enclosing block than it would have been alive for at least as long as the returned reference. While this does reduce dangling, it does not eliminate it because if the reference out lives its containing block such as by returning then dangling would still occur. These remaining dangling would at least be more visible as they are usually associated with returns, so you know where to look and if we make the proposed changes than there would be far fewer dangling to look for. It should also be noted that **the current lifetime rules of temporaries are like constants, contrary to programmer’s expectations**. This becomes more apparent with slightly more complicated examples.

“Returned References to Temporaries” [1:9]

“Similar problems already exists with references.” [1:10]

“A trivial example would be the following:” [1:11]

```
struct X { int a, b; };
int& f(X& x) { return x.a; } // return value lifetime bound to parameter
```

If `f` was called with a temporary than it too would dangle.

```
int& a = f({4, 2});  
a = 5; // fatal runtime error
```

If the lifetime of the temporary, `{4, 2}`, was bound to the lifetime of its containing block instead of its containing statement than `a` would not immediately dangle. Further, `{4, 2}` is constant initialized, so if function `f`'s signature was changed to be `int& f(const X& x)`, since it does not change `x`, then this example would not dangle at all with this proposal.

"Class `std::string` provides such an interface in the current C++ runtime library. For example:"
[1:12]

```
char& c = std::string{"hello my pretty long string"}[0];  
c = 'x'; // fatal runtime error  
std::cout << "c: " << c << '\n'; // fatal runtime error
```

Again, if the lifetime of the temporary, `std::string{"hello my pretty long string"}`, was bound to the lifetime of its containing block instead of its containing statement than `c` would not immediately dangle. Further, this more complicated compound temporary expression better illustrates why the current lifetime rules of temporaries are contrary to programmer's expectations. First of all, let's rewrite the example, as a programmer would, adding names to everything unnamed.

```
auto anonymous = std::string{"hello my pretty long string"};  
char& c = anonymous[0];  
c = 'x'; // fatal runtime error  
std::cout << "c: " << c << '\n'; // fatal runtime error
```

Even though, the code is the same from a programmer's perspective, the latter does not dangle while the former do. **Should just naming temporaries, thus turning them into variables, fix memory issues? Should just leaving variables unnamed as temporaries introduce memory issues?** Again, contrary to programmer's expectations. If we viewed unnecessary/superfluous/immediate dangling as overhead, then the current rules of temporary and constant initialization could be viewed as violations of the zero-overhead principle since just naming temporaries is reasonably written better by hand.

"There are more tricky cases like this. For example, when using the range-base for loop." [1:13]

```
for (auto x : reversed(make_vector()))
```

“with one of the following definitions, either:” [1:14]

```
template<Range R>
reversed_range reversed(R&& r) {
    return reversed_range{r}; // return value lifetime bound to parameter
}
```

“or” [1:15]

```
template<Range R>
reversed_range reversed(R r) {
    return reversed_range{r}; // return value lifetime bound to parameter
}
```

Yet again, if the lifetime of the temporary, `reversed(make_vector())`, was bound to the lifetime of its containing block instead of its containing statement than `x` would not immediately dangle. Before adding names to everything unnamed, we must expand the range based for loop.

```
{// containing block
    auto&& rg = reversed(make_vector());
    auto pos = rg.begin();
    auto end = rg.end();
    for ( ; pos != end; ++pos ) {
        auto x = *pos;
        ...
    }
}
```

Now, let's rewrite that expansion, as a programmer would, adding names to everything unnamed.

```
{// containing block
    auto anonymous1 = make_vector();
    auto anonymous2 = reversed(anonymous1);
    auto pos = anonymous2.begin();
    auto end = anonymous2.end();
    for ( ; pos != end; ++pos ) {
        auto x = *pos;
        ...
    }
}
```

Like before, the named version doesn't dangle and as such binding the lifetime of the temporary to the containing block makes more sense to the programmer than binding the lifetime of the temporary to the containing statement. In essence, from a programmer's perspective, **temporaries are anonymously named variables**.

It should be noted too that the current rules of temporaries discourages the use of temporaries because of the dangling it introduces. However, if the lifetime of temporaries was increased to a reasonable degree than programmers would use temporaries more. This would reduce dangling further because there would be fewer named variables that could be propagated outside of their containing scope. This would also improve code clarity by reducing the number of lines of code allowing any remaining dangling to be more clearly seen.

"Finally, such a feature would also help to ... fix several bugs we see in practice:" [1:16]

"Consider we have a function returning the value of a map element or a default value if no such element exists without copying it:" [1:17]

```
const V& findOrDefault(const std::map<K,V>& m, const K& key, const V& defvalue);
```

"then this results in a classical bug:" [1:18]

```
std::map<std::string, std::string> myMap;  
const std::string& s = findOrDefault(myMap, key, "none"); // runtime bug if key no
```

Is this really a bug? With this proposal, it isn't! Here is why. The function `findOrDefault` **expects** a `const string&` for its third parameter. Since C++20, `string`'s constructor is `constexpr`. It **CAN** be constructed as a constant expression. Since all the arguments passed to this `constexpr` constructor are constant expressions, in this case `"none"`, the temporary `string defvalue` **IS** also constant-initialized [2:3] (7.7 Constant expressions [expr.const]). This paper advises that if you have a non mutable `const` that it is constant-initialized, that the variable or temporary undergoes constant initialization [2:4] (6.9.3.2 Static initialization [basic.start.static]). In other words it has implicit static storage duration. The temporary would actually cease to be a temporary. As such this usage of `findOrDefault` **CAN'T** dangle.

What if `defvalue` can't be constant-initialized because it was created at runtime. If the temporary `string`'s lifetime was bound to the containing block instead of the containing statement than the chance of dangling is greatly reduced and also made more visible. You can say that it **CAN'T** immediately dangle. However, dangling still could occur if the programmer manually propagated the returned value that depends upon the temporary outside of the containing scope.

While using the containing's scope instead of the statement's scope is a vast improvement. We can actually do a little bit better. Following is an example of uninitialized and delayed initialization.

```
bool test();

struct X { int a, b; };

constexpr const X* ref2pointer(const X& ref)
{
    return &ref;
}

X x_factory(int a, int b)// not constexpr, runtime construction
{
    return {a, b};
}

int main()
{
    const X* x;// uninitialized
    if(test())
    {
        x = ref2pointer({2, 4});// implicit constant initialization
    }
    else
    {
        x = ref2pointer(x_factory(4, 2));// delayed initialization
        // statement scope or
        // containing scope or
        // scope of the variable to which the temporary is assigned
    }
}
```

According to this proposal, `ref2pointer({2, 4})` would undergo implicit constant initialization so that expression would not dangle.

The variable `x` would dangle if initialized with the expression `ref2pointer(x_factory(4, 2))` when the scope is bound to the containing statement. The variable would also dangle if initialized with the expression `ref2pointer(x_factory(4, 2))` when the scope is bound to the containing block. The variable would NOT dangle if initialized with the expression `ref2pointer(x_factory(4, 2))` when the scope is bound to the lifetime of the variable to which the temporary is assigned, in this case `x`.

Extending the lifetime of the temporary to be the lifetime of the variable to which it is assigned is not unreasonable for C++. Matter of fact it is already happening but the rules are so restrictive that it limits its use by many programmers as the following examples illustrate.

Working Draft, Standard for Programming Language C++ [2:5]

“6.7.7 Temporary objects”

...

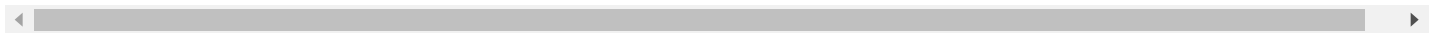
“₅ There are ... contexts in which temporaries are destroyed at a different point than the end of the full expression.”

...

“(6.8)”

```
template<typename T> using id = T;

int i = 1;
int&& a = id<int[3]>{1, 2, 3}[i]; // temporary array has same lifetime as a
const int& b = static_cast<const int&>(0); // temporary int has same lifetime as b
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
// exactly one of the two temporaries is lifetime-extended
```



```
const int& x = (const int&)1; // temporary for value 1 has same lifetime as x
```

```
struct S {
    const int& m;
};
const S& s = S{1}; // both S and int temporaries have lifetime of s
```

The preceding sections of this proposal is identical at times in wording, in structure as well as in examples to p0936r0, the Bind Returned/Initialized Objects to the Lifetime of Parameters [1:19] proposal. This shows that similar problems can be solved with simpler solutions, that programmers are already familiar with, such as constants and naming temporaries. It must be conceded that Bind Returned/Initialized Objects to the Lifetime of Parameters [1:20] is a more general solution that fixes more dangling while this proposal is more easily understood by programmers of all experience levels but fixes less dangling.

Why not just extend the lifetime as prescribed in `Bind Returned/Initialized Objects to the Lifetime of Parameters` ?

In that proposal, a question was raised.

“Lifetime Extension or Just a Warning?” “We could use the marker in two ways:”

1. *“Warn only about some possible buggy behavior.”*
2. *“Fix possible buggy behavior by extending the lifetime of temporaries”*

In reality, there are three scenarios; warning, **error** or just fix it by extending the lifetime.

However, things in the real world tend to be more complicated. Depending upon the scenario, at least theoretically, some could be fixed, some could be errors and some could be warnings. Further, waiting on a more complicated solution that can fix everything may never happen or worse be so complicated that the developer, who is ultimately responsible for fixing the code, can no longer understand the lifetimes of the objects created. Shouldn't we fix what we can, when we can; i.e. low hanging fruit. Also, fixing everything the same way would not even be desirable. Let's consider a real scenario. Extending one's lifetime could mean 2 different things.

1. Change automatic storage duration such that a instances' lifetime is just moved lower on the stack as prescribed in `p0936r0`.
2. Change automatic storage duration to static storage duration. [This is what I am proposing but only for those that it logically applies to.]

If only #1 was applied holistically via `p0936r0`, `-wlifetime` or some such, then that would not be appropriate or reasonable for those that really should be fixed by #2. Likewise #2 can't fix all but DOES make sense for those that it applies to. As such, this proposal and `p0936r0` ^[1:21] are complimentary.

Personally, `p0936r0` ^[1:22] or something similar should be adopted regardless because we give the compiler more information than it had before, that a return's lifetime is dependent upon argument(s) lifetime. When we give more information, like we do with `const` and `constexpr`, the `c++` compiler can do amazing things. Any reduction in undefined behavior, dangling references/pointers and delayed/uninitialized errors should be welcomed, at least as long it can be explained simply and rationally.

Proposed Wording

6.7.5.4 Automatic storage duration [`basic.stc.auto`]

¹ Variables that belong to a block or parameter scope and are not explicitly declared static, `thread_local`, `extern` or had not underwent implicit constant initialization (6.9.3.2) have automatic storage duration. The storage for these entities lasts until the block in which they are created exits.

...

6.7.7 Temporary objects

...

⁴ When an implementation introduces a temporary object of a class that has a non-trivial constructor (11.4.5.2, 11.4.5.3), it shall ensure that a constructor is called for the temporary object. Similarly, the destructor shall be called for a temporary with a non-trivial destructor (11.4.7). Temporary objects are destroyed via automatic storage duration (6.7.5.4) associated with the enclosing block of the expression as if the compiler was naming the temporaries anonymously or via automatic storage duration associated with the enclosing block of the variable to which the temporary is assigned, whichever is greater lifetime as the last step in evaluating the full expression (6.9.1) that (lexically) contains the point where they were created. This is true even if that evaluation ends in throwing an exception. The value computations and side effects of destroying a temporary object are associated only with the full expression, not with any specific subexpression.

⁵ ~~There are three contexts in which temporaries are destroyed at a different point than the end of the full expression. The first context is when a default constructor is called to initialize an element of an array with no corresponding initializer (9.4). The second context is when a copy constructor is called to copy an element of an array while the entire array is copied (7.5.5.3, 11.4.5.3). In either case, if the constructor has one or more default arguments, the destruction of every temporary created in a default argument is sequenced before the construction of the next array element, if any.~~

⁶ ~~The third context is when a reference binds to a temporary object.²⁹ The temporary object to which the reference is bound or the temporary object that is the complete object of a subobject to which the reference is bound persists for the lifetime of the reference if the glvalue to which the reference is bound was obtained through one of the following:~~

~~(6.1) — a temporary materialization conversion (7.3.5);~~

~~(6.2) — (expression), where expression is one of these expressions;~~

~~(6.3) — subscripting (7.6.1.2) of an array operand, where that operand is one of these expressions;~~

~~(6.4) — a class member access (7.6.1.5) using the . operator where the left operand is one of these expressions and the right operand designates a non-static data member of non-reference type;~~

~~(6.5) — a pointer to member operation (7.6.4) using the .* operator where the left operand is one of these expressions and the right operand is a pointer to data member of non-reference type;~~

~~(6.6) — a~~

~~(6.6.1) — const_cast (7.6.1.11);~~

~~(6.6.2) — static_cast (7.6.1.9);~~

~~(6.6.3) — dynamic_cast (7.6.1.7); or~~

~~(6.6.4) — reinterpret_cast (7.6.1.10)~~

~~converting, without a user-defined conversion, a glvalue operand that is one of these expressions to a glvalue that refers to the object designated by the operand, or to its complete object or a subobject thereof;~~

~~(6.7) — a conditional expression (7.6.16) that is a glvalue where the second or third operand is one of these expressions; or~~

~~(6.8) — a comma expression (7.6.20) that is a glvalue where the right operand is one of these expressions.~~

[Example 2:

```
template<typename T> using id = T;  
  
int i = 1;  
int&& a = id<int[3]>{1, 2, 3}[i]; // temporary array has same lifetime as a  
const int& b = static_cast<const int&>(0); // temporary int has same lifetime as b  
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);  
// exactly one of the two temporaries is lifetime-extended
```

~~—end example]~~

~~[Note 5: An explicit type conversion (7.6.1.4, 7.6.3) is interpreted as a sequence of elementary casts, covered above.~~

[Example 3:

```
const int& x = (const int&)1; // temporary for value 1 has same lifetime as x
```

~~—end example}~~

~~—end note}~~

~~[Note 6: If a temporary object has a reference member initialized by another temporary object, lifetime extension applies recursively to such a member's initializer.~~

~~[Example 4:~~

```
struct S {  
—const int& m;  
};  
const S& s = S{1}; // both S and int temporaries have lifetime of s
```

~~—end example}~~

~~—end note}~~

~~The exceptions to this lifetime rule are:~~

~~(6.9) — A temporary object bound to a reference parameter in a function call (7.6.1.3) persists until the completion of the full expression containing the call.~~

~~(6.10) — A temporary object bound to a reference element of an aggregate of class type initialized from a parenthesized expression list (9.4) persists until the completion of the full expression containing the expression list.~~

~~(6.11) — The lifetime of a temporary bound to the returned value in a function return statement (8.7.4) is not extended; the temporary is destroyed at the end of the full expression in the return statement.~~

~~(6.12) — A temporary bound to a reference in a new initializer (7.6.2.8) persists until the completion of the full expression containing the new initializer.~~

~~[Note 7: This might introduce a dangling reference. —end note}~~

~~[Example 5:~~

```
struct S { int mi; const std::pair<int,int>& mp; };  
S a { 1, {2,3} };  
S* p = new S{ 1, {2,3} }; // creates dangling reference
```

—end example]

~~z~~ The destruction of a temporary whose lifetime is not extended by being bound to a reference is sequenced before the destruction of every temporary which is constructed earlier in the same full expression. If the lifetime of two or more temporaries to which references are bound ends at the same point, these temporaries are destroyed at that point in the reverse order of the completion of their construction. In addition, the destruction of temporaries bound to references shall take into account the ordering of destruction of objects with static, thread, or automatic storage duration (6.7.5.2, 6.7.5.3, 6.7.5.4); that is, if obj1 is an object with the same storage duration as the temporary and created before the temporary is created the temporary shall be destroyed before obj1 is destroyed; if obj2 is an object with the same storage duration as the temporary and created after the temporary is created the temporary shall be destroyed after obj2 is destroyed.

~~g~~ [Example 6:

```
struct S {  
—S();  
—S(int);  
—friend S operator+(const S&, const S&);  
—~S();  
};  
S obj1;  
const S& cr = S(16)+S(23);  
S obj2;
```

The expression `S(16) + S(23)` creates three temporaries: a first temporary T1 to hold the result of the expression `S(16)`, a second temporary T2 to hold the result of the expression `S(23)`, and a third temporary T3 to hold the result of the addition of these two expressions. The temporary T3 is then bound to the reference `cr`. It is unspecified whether T1 or T2 is created first. On an implementation where T1 is created before T2, T2 shall be destroyed before T1. The temporaries T1 and T2 are bound to the reference parameters of `operator+`; these temporaries are destroyed at the end of the full expression containing the call to `operator+`. The temporary T3 bound to the reference `cr` is destroyed at the end of `cr`'s lifetime, that is, at the end of the program. In addition, the order in which T3 is destroyed takes into account the destruction order of other objects with static storage duration. That is, because `obj1` is constructed before T3, and T3 is constructed before `obj2`, `obj2` shall be destroyed before T3, and T3 shall be destroyed before `obj1`.

—end example]

6.9.3.2 Static initialization [basic.start.static]

...

² Constant initialization is performed explicitly if a variable or temporary object with static or thread storage duration is constant-initialized (7.7). Constant initialization is performed implicitly if a non mutable const variable or non mutable const temporary object is constant-initialized (7.7). If constant initialization is not performed, a variable with static storage duration (6.7.5.2) or thread storage duration (6.7.5.3) is zero-initialized (9.4). Together, zero-initialization and constant initialization are called static initialization; all other initialization is dynamic initialization. All static initialization strongly happens before (6.9.2.2) any dynamic initialization.

In Depth Rationale

There is a general expectation across programming languages that constants or more specifically constant literals are “immutable values which are known at compile time and do not change for the life of the program”. ^[3] In most programming languages or rather the most widely used programming languages, constants do not dangle. Constants are so simple, so trivial (English wise), that it is shocking to even have to be conscience of dangling. This is shocking to c++ beginners, expert programmers from other programming languages who come over to c++ and at times even shocking to experienced c++ programmers.

Why not before

There is greater need now that more types are getting constexpr constructors. Also types that would normally only be dynamically allocated, such as string and vector, since c++20, can also be constexpr. This has opened up the door wide for many more types being constructed at compile time.

Storage Duration

Working Draft, Standard for Programming Language C++ ^[2:6]

“5.13.5 String literals [lex.string]”

“⁹ **Evaluating a string-literal results in a string literal object with static storage duration** (6.7.5). *Whether all string-literals are distinct (that is, are stored in nonoverlapping objects) and whether successive evaluations of a string-literal yield the same or a different object is unspecified.*”

“[Note 4: The effect of attempting to modify a string literal object is undefined. — end note]”

String literals have static storage duration, and thus exist in memory for the life of the program. All the other types of literals have automatic storage duration by default. While that may makes perfect sense for literals that are not constant, constants on other hand are generally believed to be the same value for the life of the program and are ideal candidates to have static storage duration so they can exist in memory for the life of the program. Since literals currently don't behave this way, constant [like] literals are not as simple as they could be, leading to superfluous dangling as well as language inconsistencies.

non dangling	dangling
<pre>const char* non_dangling_42() { const char* constant = "42"; return constant; }</pre>	<pre>const int& dangl { const int co //static con return const }</pre>
<pre>const char* maybe_dangling_42(const char* maybe_constant) { return maybe_constant; } maybe_dangling_42("42");// NOT dangling</pre>	<pre>const int& maybe { return maybe } // dangling beca // 42 is auto no // 42's lifetime maybe_dangling_4</pre>

Even though the usage is the same, the constant string examples do not dangle because they have static storage duration while the dangling, non string literal has automatic storage duration.

Constant Expressions

C++ Core Guidelines: Programming at Compile Time with constexpr [4]

“A constant expression”

- *“**can** be evaluated at **compile time**.”*
- *“give the compiler deep insight into the code.”*
- *“are implicitly thread-safe.”*
- *“**can** be constructed in the **read-only memory (ROM-able)**.”*

It isn't just that resolved constant expressions **can** be placed in ROM which makes programmers believe these **should** be stored globally but also the fact that fundamentally **these expressions are executed at compile time**. Along with templates, constant expressions are the closest thing c++ has to **pure** functions. That means the results are the same given the parameters, and since these expressions run at compile time, than the resultant values are the same, no matter where or when in the c++ program. This is essentially global to the program; technically across programs too.

This proposal just requests, at least in specific scenarios, that instead of resolved constant-initialized constant expressions **CAN** be ROMable but rather that they **HAVE** to be or at least the next closest thing; constant and static storage duration .

Constant Initialization

Working Draft, Standard for Programming Language C++ [2:7]

“6.9.3.2 Static initialization [basic.start.static]”

“₁ Variables with static storage duration are initialized as a consequence of program initiation. Variables with thread storage duration are initialized as a consequence of thread execution. Within each of these phases of initiation, initialization occurs as follows.”

“₂ Constant initialization is performed if a variable or temporary object with static or thread storage duration is constant-initialized (7.7).”

The syntax for constant initialization is as follows:

```
static T & ref = constexpr; // constant-initialized required
```

```
static T object = constexpr; // constant-initialized required
```

For the sake of this proposal, I am currently only talking about a subset of `constant initialization` where `T` is `const` since the primary focus is on constants.

```
static const T & ref = constexpr; // constant-initialized required
```

```
static const T object = constexpr; // constant-initialized required
```

Ironically, at namespace scope, variables are already implicitly static.

Working Draft, Standard for Programming Language C++ [2:8]

“6.7.5.2 Static storage duration [*basic.stc.static*]”

“All variables which”

- *“do not have thread storage duration and”*
- *“belong to a **namespace scope** (6.4.5) or are first declared with the **static** or **extern** keywords (9.2.2) have static storage duration. The storage for these entities lasts for the duration of the program (6.9.3.2, 6.9.3.4).”*

As such, the previous example could simply be written as the following:

```
const T & ref = constexpr; // constant-initialized required
```

```
const T object = constexpr; // constant-initialized required
```

Unfortunately, the static storage duration doesn't come from the fact that it is a constant-initialized constant expression and that a constant was expected/requested. Consider for a moment, if it did, that is implicit static storage duration by looking at this from the perspective of constant definitions in class definition, function body and parameters/arguments.

constant definition in class definitions

```

struct S
{
    const T & ref = constexpr;
    const T object = constexpr;
};

```

If the class members `ref` and `object` were implicitly made static the result to class `S` would be a reduction in size and initialization time. The class members `ref` and `object` would still be `const` and their respective types, so their should be no change in the logic of the code that directly depends upon them. While supporting implicit static storage duration here does not decrease dangling, it would be of benefit from a language simplicilty standpoint for this to be consistent with the next two sections; function body and their parameters/arguments. However since this section has to do with class data member definition and not code execution, explicit static could still be required, if their is a real and significant concern over code breakage.

constant definition in function body

```

const int& dangling_42()
{
    const int& constant = 42;
    return constant;
}

const int& dangling_42()
{
    const int constant = 42;
    return constant;
}

const int& dangling_42()
{
    return 42;
}

```

With implicit static storage duration, these examples would not dangle and would make more sense. The local variable `constant` and even unnamed variable, temporary, `42`, still is `const` and have their respective types, so their should be no change in the logic of the code that directly depends upon them. As such, this feature isn't expected to break existing code.

constant definition in function parameter and arguments

```
const int& maybe_dangling_42(const int& maybe_constant);

maybe_dangling_42(42);
```

This is expected to be a non breaking change because a parameter that takes a `const&` argument doesn't know whether the instance was created locally, statically or even dynamically. The advantage of adding implicit constant initialization here would be that those temporary `const` argument scenarios would no longer dangle. Static storage duration for arguments is actually the primary goal of this paper. It should also be noted that while `static` can be applied explicitly in class data member definition and in function bodies, `static` isn't even an option as a modifier to a function argument, so the user doesn't have a choice and the current default of automatic storage duration instead of static storage duration is less intuitive when constants or constant expressions are involved. Even if the keyword `static` could be applied to arguments, programmer's code will be littered with that keyword as they will start applying it to every argument initialized in a constant expression fashion.

Impact on current proposals

p2255r2

A type trait to detect reference binding to temporary ^[5]

Following is a slightly modified `constexpr` example taken from the p2255r2 ^[5:1] proposal. Only the suffix `s` has been added. It is followed by a non `constexpr` example. Currently, such an example is immediately dangling. Via p2255r2 ^[5:2], both examples become ill formed. However, with this proposal the `constexpr` example becomes valid.

	constexpr	runtime
Examples	<code>std::tuple<const std::string&> x("hello"s);</code>	<code>std::tuple<const s</code>
Before	<code>// dangling</code>	<code>// dangling</code>
p2255r2 [5:3]	<code>// ill-formed</code>	<code>// ill-formed</code>
p2255r2 [5:4] and this proposal	<code>// correct</code>	<code>// ill-formed</code>
this proposal only	<code>// correct</code>	<code>// well-formed but</code>

The proposed valid example is reasonable from many programmers perspective because "hello"s is a literal just like "hello" is a safe literal in C++. Implicit constant initialization turns constant temporaries into just global variables as such they are no longer temporaries and such would not impact this proposal.

C99 compound literals are **safer** literals because the lifetime is the life of the block containing the temporary instead of the expression. As such turning temporaries into anonymously named variables would remove the need to use this trait to prevent dangling since their would be valid mutable usage of temporaries that no longer immediately dangle.

p2576r0

The constexpr specifier for object definitions ^[6]

The p2576r0 ^[6:1] proposal is about contributing constexpr back to the c programming language. Interestingly, C++ has constexpr in the first place, in part, to allow c99 compound literals in C++. In the p2576r0 ^[6:2] proposal there are numerous references to “constant

expression” and “static storage duration” highlighting that this and my proposal are playing in the same playground. Consider the following:

“C requires that objects with static storage duration are only initialized with constant expressions.

“Because C limits initialization of objects with static storage duration to constant expressions, it can be difficult to create clean abstractions for complicated value generation.”

Further, the p2576r0 [6:3] proposal has a whole section devoted to just storage duration.

“3.4. Storage duration”

“For the storage duration of the created objects we go with C++ for compatibility, that is per default we have automatic in block scope and static in file scope. The default for block scope can be overwritten by static or refined by register. It would perhaps be more natural for named constants”

- “to be addressless (similar to a register declaration or an enumeration),”*
- “to have static storage duration (imply static even in block scope), or”*
- “to have no linkage (similar to typedef or block local static)”*

“but we decided to go with C++’s choices for compatibility.”

My proposal would constitute a delay in the p2576r0 [6:4] proposal as I am advocating for refining the c++ choices before contributing constexpr back to c. I also believe that the p2576r0 [6:5] proposal fails to consider a fourth alternative with respect to storage duration and that is to go with how c handles compound literals, improve upon it and have c++ to conform with it. That scenario is the second feature proposed by this proposal.

Past

A brief consideration of the proposals that led to constexpr landing as a feature in c++11 bears weight on the justification of refining constant initialization.

n1511

2003

*“This note proposes a notion of **user-defined literals** based on literal constructors without requiring new syntax. If combined with the separate proposal for generalized initializer lists, it becomes a generalization of the **C99 notion of compound literals**.”*

*“However, a constructor is a very general construct and there have been many requests for a way to express literals for user-defined types in such a way that a programmer can be **confident that a value will be constructed at compile time and potentially stored in ROM**. For example:”*

```
complex z(1,2); // the variable z can be constructed at compile time
const complex cz(1,2); // the const cz can potentially be put in ROM
```

*“**Personally, I prefer (1): basically, a value is a literal if it is composed out of literals and implemented by a literal constructor. The problem with that is that some people will not trust compilers to do proper resolution, placement in ROM, placement in text segment**”*

*“**C99 compound literals**”*

“In C99, it is explicitly allowed to take the address of a compound literal. For example:”

```
f(&(struct foo) { 1,2 } );
```

*“**This makes sense only if we assume that the {1,2} is stored in a data segment (like a string literal, but different from a int literal). I see no problem allowing that**”*

*“**It would be tempting to expand this rule to user-defined literals bound to references.**”*

The other reason why we should re-evaluate user-defined literals bound to references is to reduce dangling references and even dangling pointers. It is also surprising that user defined literals do not work simply without memory issues and that they currently work better in C and practically in every other language than it does in C++. ROM, read only memory, is effectively global/static storage duration and const. Note too that the original motivation for constant expressions are for literals. So constant expressions are effectively literals. Note also the following example provided in that proposal.

```
complex z(1,2); // the variable z can be constructed at compile time  
const complex cz(1,2); // the const cz can potentially be put in ROM
```

While both constant-initialized constant expressions are the same, the detail that decides whether it is a literal that can even be placed in ROM was the `const` keyword. So, `const` constant-initialized constant expressions are constant literals or simply, constants.

n2235

2007

*“This paper generalizes the notion of constant expressions to include constant-expression functions and **user-defined literals**”*

*“The goal is ... to **increase C99 compatibility.**”*

...

*“**simplify the language** definition in the area of constant expression **to match existing practice**”*

...

*“**3. Static initialization of objects with static storage.**”*

*“... However **we strive for more uniform and consistency among related C++ language features and compatibility**”*

*“**3 Problems**”*

...

*“**3.4 Unexpected dynamic initialization**”*

*“**However, it is possible to be surprised by expressions that (to someone) “look const” but are not.**”*

*“**3.5 Complex rules for simple things**”*

*“The focus of this proposal is to address the issues mentioned in preceding sections. However, discussions in the Core Working Group at the Berlin meeting (April 2006) concluded that the current rules for integral constant expressions are too complicated, and source of several Defect Reports. **Consequently, a “cleanup”, i.e. adoption of simpler, more general rules is suggested.**”*

...

“As for other const variables, storage need not be allocated for a constant expression datum, unless its address is taken.”

```
// the &x forces x into memory
```

“When the initializer for an ordinary variable (i.e. not a constexpr) happens to be a constant, the compiler can choose to do dynamic or static initialization (as ever).”

“Declaring a constructor constexpr will help compilers to identify static initialization and perform appropriate optimizations (like putting literals in read-only memory.) Note that since “ROM” isn’t a concept of the C++ Standard and what to put into ROM is often a quite subtle design decision, this proposal simply allows the programmer to indicate what might be put into ROM (constant-expression data) rather than trying to specify what actually goes into ROM in a particular implementation.”

“We do not propose to make constexpr a storage-class-specifier because it can be combined with either static or extern or register, much like const.”

Most of these references are given to give us a better idea of the current behavior of constexpr. However, it should be noted that the motivations of this proposal is similar to the motivations for constexpr itself. Mainly ...

- Increase C99 compatibility
- Simplify the language to match existing practice
- Lessen the surprise of unreasonable lifetimes by expressions that look and are const
- Consequently, a “cleanup”, i.e. adoption of simpler, more general rules

Present

This proposal should also be considered in the light of the current standards. A better idea of our current rules is necessary to understanding how they may be simplified for the betterment of C++ .

C Standard Compound Literals

Let’s first look at how literals specifically compound literals behave in C . There is still a gap between C99 and C++ and closing or reducing that gap would not only increase our compatibility but also reduce dangling.

“6.5.2.5 Compound literals”

paragraph 5

*“The value of the compound literal is that of an unnamed object initialized by the initializer list. If the compound literal occurs outside the body of a function, the object has static storage duration; otherwise, it has **automatic storage duration associated with the enclosing block.**”*

The lifetime of this “enclosing block” is longer than that of `c++`. In `c++` under 6.7.7 Temporary objects [class.temporary] specifically 6.12 states a *temporary bound to a reference in a new-initializer (7.6.2.8) persists until the completion of the full-expression containing the new-initializer.*

gcc [10] describes the result of this gap.

“In C, a compound literal designates an unnamed object with static or automatic storage duration. In C++, a compound literal designates a temporary object that only lives until the end of its full-expression. As a result, well-defined C code that takes the address of a subobject of a compound literal can be undefined in C++, so G++ rejects the conversion of a temporary array to a pointer.”

Simply put `c` has fewer dangling than `c++` ! What is more is that `c`’s solution covers both `const` and non `const` temporaries! Even though it is `c`, it is more like `c++` than what people give this feature credit for because it is tied to blocks/braces, just like RAIL. This adds more weight that the `c` way is more intuitive. Consequently, the remaining dangling should be easier to spot for developers not having to look at superfluous dangling.

GCC even takes this a step forward which is closer to what this proposal is advocating. The last reference also says the following.

“As a GNU extension, GCC allows initialization of objects with static storage duration by compound literals (which is not possible in ISO C99 because the initializer is not a constant). It is handled as if the object were initialized only with the brace-enclosed list if the types of the compound literal and the object match. The elements of the compound literal must be constant. If the object being initialized has array type of unknown size, the size is determined by the size of the compound literal.”

Even the `c++` standard recognized that there are other opportunities for constant initialization.

“6.9.3.2 Static initialization [basic.start.static]”

“3 An implementation is permitted to perform the initialization of a variable with static or thread storage duration as a static initialization even if such initialization is not required to be done statically, provided that”

“(3.1) — the dynamic version of the initialization does not change the value of any other object of static or thread storage duration prior to its initialization, and”

“(3.2) — the static version of the initialization produces the same value in the initialized variable as would be produced by the dynamic initialization if all variables not required to be initialized statically were initialized dynamically.”

This proposal is one such opportunity. Besides improving constant initialization, we'll be increasing memory safety by reducing dangling.

Should c++ just adopt c99 literal lifetimes being scoped to the enclosing block instead of to the c++ statement, in lieu of this proposal?

NO, there is still the expectation among programmers that constants, const evaluations of const-initialized constant expressions, are of static storage duration.

Should c++ adopt c99 literal lifetimes being scoped to the enclosing block instead of to the c++ statement, in addition to this proposal?

YES, c99 literal lifetimes does not guarantee any reduction in dangling, it just reduces it. This proposal does guarantee but only for const evaluations of constant-initialized constant expressions. Combined their would be an even greater reduction in dangling. As such this proposal and c99 compound literals are complimentary. The remainder can be mitigated by other measures.

Should c++ adopt c99 literal lifetimes being scoped to the enclosing block instead of to the c++ statement?

YES, the c++ standard is currently telling programmers that the first two examples in the following table are equivalent with respect to the lifetime of the temporary `{1, 2}` and the named variable `cz`. This is because the lifetime of the temporary `{1, 2}` is bound to the statement, which means it is destroyed before `some_code_after` is called.

Given

```
void any_function(const complex& cz);
```

Programmer code	What c++ is actually doing!	Programme
<pre>void main() { some_code_before(); any_function({1, 2}); some_code_after(); }</pre>	<pre>void main() { some_code_before(); { const complex anonymous{1, 2}; any_function(anonymous); } some_code_after(); }</pre>	<pre>void main { some_cc const c any_fun some_cc }</pre>

This is contrary to general programmer expectations and how it behaves in c99 . Besides the fact that a large portion of the c++ community has their start in c and besides the fact that no one, in their right mind, would ever litter their code with superfluous braces for every variable that they would like to be a temporary, there is a more fundamental reason why it is contrary to general programmer expectations. It can actually be impossible to write it that way. Consider another example, now with a return value in which the type does not have a default constructor.

Given

```
no_default_constructor any_function(const complex& cz);
```

Programmer code	<pre> void main() { some_code_before(); no_default_constructor ndc = any_function({1, 2}); some_code_after(ndc); } </pre>
What is c++ doing?	<pre> void main() { some_code_before(); { const complex anonymous{1, 2}; no_default_constructor ndc = any_function(anonymous); } some_code_after(ndc); } </pre>
What is c++ doing?	<pre> void main() { some_code_before(); no_default_constructor ndc; { const complex anonymous{1, 2}; ndc = any_function(anonymous); } some_code_after(ndc); } </pre>

It should be noted that neither of the “ what is C++ doing? ” examples even compile. The first because the variable `ndc` is not accessible to the functional call `some_code_after` . The second because the class `no_default_constructor` doesn’t have a default constructor and as such does not have a uninitialized state. In short, the current `c++` behavior of statement scoping of temporaries instead of containing block scoping is more difficult to reason about because the equivalent code cannot be written by the programmer. As such the `c99` way is simpler, safer and more reasonable.

The fundamental flaw

Consider for a moment if the C++ rules were that all variables, named or unnamed/temporaries, ***persists until the completion of the full-expression containing the new-initializer.*** ^[2:10]

How useful would that be?

```
auto s = "hello world"s; // immediate dangling reference
std::string_view sv = "hello world"s; // immediate dangling reference
auto reference = some_function("hello world"s);
use_the_ref(reference); // dangle
```

The variable `s` would not be usable. All variables would mostly be immediately dangling. The variable `s` could not be used safely by any statements that follow its initialization. It could not be used safely in nested blocks that follow be that `if`, `for` and `while` statements to name a few. The only place the variable could be used safely if it was anonymously passed as a argument to a function. That would allow multiple statements inside the function call to make use of the instance. If the function returned a reference to the argument or any part of it than there would be further dangling even though it is not unreasonable for a function to return a reference to a portion of or a whole instance, especially when the instance is known to already be alive lower on the stack. In essence, such a rule **divorces the lifetime of the instance from the variable name**. The only use of this from a programmer's perspective is the anonymity of not naming variables as a form of access control. In short, programmers could not program. Doesn't this sound familiar, for it is our current temporary lifetime rule!

Now, consider for a moment if the C++ rules were that all variables that do not have static storage duration, has automatic storage duration associated with the enclosing block of the expression as if the compiler was naming the temporaries anonymously or associated with the enclosing block of the variable to which the initialization is assigned, whichever is greater lifetime. How useful would that be?

```
auto s = "hello world"s;
std::string_view sv = "hello world"s;
auto reference = some_function("hello world"s);
use_the_ref(reference);
```

The variable `s` would be usable. No variables would immediately dangle. The variable `s` could be used safely by any statements that follow its initialization. It could be used safely in nested blocks that follow be that `if`, `for` and `while` statements to name a few. By default, the variable could be used safely when anonymously passed as a argument to a function. If the function returned a reference to the argument or any part of it than there would not be further dangling unless the developer manually propagated the reference lower on the stack such as with a `return`. Even the benefit of anonymity when using temporaries are not lost and the longer

lifetime doesn't impact other instances that don't even have access to said temporary. In short, programmers are freed from much dangling. Further, much the remaining dangling coalesces around returns and yields. This is what is proposed.

C++ Standard

It is also good to consider how the `c++` standard impacts this proposal and how the standard may be impacted by such a proposal.

Working Draft, Standard for Programming Language C++ [2:11]

String literals are traditionally one of the most common literals and in `c++` they have static storage duration. This is also the case in many programming languages. As such, these facts leads developers to incorrectly believe that all literals or at least all constant literals have static storage duration.

"5.13.5 String literals [lex.string]"

*"9 Evaluating a string-literal results in a string literal object with **static storage duration** (6.7.5). Whether all string-literals are distinct (that is, are stored in nonoverlapping objects) and whether successive evaluations of a string-literal yield the same or a different object is unspecified."*

*"[Note 4: **The effect of attempting to modify a string literal object is undefined.** — end note]"*

This proposal aligns or adjusts [constant] literals not only with `c` compound literals but also with `c++` string literals. It too should be noted that `c++` is seemingly inconsistent on whether other string like literals should behave lifetime wise like string. Are array literals of `static storage duration` ? What about if the array was of characters? What about string like literals such as `std::string` and `std::string_view`?

`c++` also says the *"effect of attempting to modify a string literal object is undefined."* With us having `const` for so long, there is few reasons left for this to go undefined. Undefined behavior doesn't make constants and non constant literals any easier to deal with. A string literal could have **static storage duration** for constant expressions and **automatic storage duration** for **non** constant expressions, just like other literals. The lifetime of the `automatic storage duration` could be the `c` rule of the enclosing block since it is safer than `c++` . This would further increase the consistency between string literals and custom/constexpr literals. However, considering that string literals currently have `static storage duration` and we want to reduce dangling instead of increasing it by making the lifetime too narrow, it would be

reasonable to include rules for uninitialized and general lifetime extension via `Bind Returned/Initialized Objects to the Lifetime of Parameters` ^[1:23] before nudging string literals closer to non string literals.

The section of the `c++` standard on temporary objects has an example of dangling.

“6.7.7 Temporary objects [class.temporary]”

“(6.12) — A temporary bound to a reference in a new-initializer (7.6.2.8) persists until the completion of the full-expression containing the new-initializer.”

“[Note 7: This might introduce a dangling reference. — end note]”

“[Example 5:]”

```
struct S { int mi; const std::pair<int,int>& mp; };  
S a { 1, {2,3} };  
S* p = new S{ 1, {2,3} }; // creates dangling reference
```

“— end example]”

It should be noted that this example is not an example of dangling in `c99` or with my proposal because the temporaries would live as long as their containing block which is sufficient for `a` and `p` in this limited example. Further, since `mp` expects a constant, `std::pair` can be constructed at compile time and was constant-initialized than implicit constant initialization could also occur.

“9.4 Initializers [dcl.init]”

Similarly, the section of the `c++` standard on initializer has multiple examples of dangling.

“9.4.1 General [dcl.init.general]”

“(16.6.2.2)”

```

struct A {
    int a;
    int&& r;
};

int f();
int n = 10;

A a1{1, f()}; // OK, lifetime is extended
A a2(1, f()); // well-formed, but dangling reference
A a3{1.0, 1}; // error: narrowing conversion
A a4(1.0, 1); // well-formed, but dangling reference
A a5(1.0, std::move(n)); // OK

```

The `a2` and `a4` initializations would no longer dangle when the temporary has automatic storage duration associated with the enclosing block as the code would be same had it been written as follows:

```

int anonymous1 = f();
A a2(1, anonymous1); // well-formed
int anonymous2 = 1;
A a4(1.0, anonymous2); // well-formed

```

“9.4.5 List-initialization [dcl.init.list]”

Similarly, the section of the `C++` standard on list-initialization has an example of dangling.

```

struct A {
    std::initializer_list<int> i4;
    A() : i4{ 1, 2, 3 } {} // ill-formed, would create a dangling reference
};

```

According to this proposal, this example would neither be ill formed or dangling as initializer lists expects a const array, arrays can be constexpr constructed and the array was constant-initialized. Thus implicit constant initialization would kick in.

“11.9.6 Copy/move elision [class.copy.elision]”

Similarly, the section of the `C++` standard on copy and move elision has examples of dangling.

```
constexpr A a; // well-formed, a.p points to a
constexpr A b = g(); // error: b.p would be dangling (7.7)
void h() {
A c = g(); // well-formed, c.p can point to c or be dangling
}
```

Not only would `b` no longer dangle but the ambiguity of `c.p` could “point to `c` or be dangling” would be gone.

Outstanding Issues

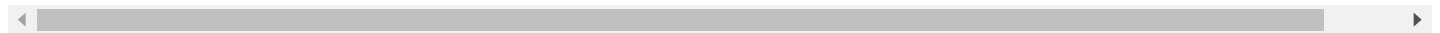
P1018R16

The C++ Language Evolution status pandemic edition ^[11] list some issues that could be effected positively by this proposal. Many of these have the NAD , Not A Defect, designation. While these may not presently have been a defect, they were surprising enough to have been brought forth in the first place.

LWG2432 `initializer_list` assignability

```
auto il1 = {1,2,3};

il1 = {7,8,9}; // currently well-formed but dangles immediately; should be ill-for
```



With implicit constant initialization and if `il1` was `const auto` instead of `auto` , this example would not dangle and as such should not be ill-formed. With c99 literal enclosing block lifetime, this example, AS IS, would not dangle.

CWG900 Lifetime of temporaries in range-based for

```

// some function

std::vector<int> foo();

// correct usage

auto v = foo();

for( auto i : reverse(v) ) { std::cout << i << std::endl; }

// problematic usage

for( auto i : reverse(foo()) ) { std::cout << i << std::endl; }

```

I am not sure what is more shocking. That this is even an issue or that the current resolution is NAD .

With implicit constant initialization ,if i was const auto instead of auto , if foo was a constexpr and if reverse was a constexpr , this example would not dangle and as such should not be ill-formed.

With c99 literal enclosing block lifetime, this example, AS IS, would not dangle.

In the identifying paper for this issue, Fix the range-based for loop, Rev1 ^[12], says the following:

“The Root Cause for the problem”

*“The reason for the undefined behavior above is that according to the current specification, the range-base for loop internally is **expanded to multiple statements**:”*

- *“First, we have some initializations using the for-range-initializer after the colon and”*
- *“Then, we are calling a low-level for loop”*

While certainly a factor, the problem is **NOT** that internally, the range-base for loop is expanded to multiple statements. It is rather that one of those statements has a scope of the statement instead of the scope of the containing block. The scoping difference between c99 and c++ rears it head again. From the programmers perspective, the issue in both cases is that c++ doesn't treat temporaries, unnamed variable as if they were named by the programmer just anonymously. The supposed correct usage highlights this fact.

```
auto v = foo();

for( auto i : reverse(v) ) { std::cout << i << std::endl; }
```

If you just name it, it works! Had `reverse(foo())` been scoped to the block that contains the range based for loop than this too would have worked.

Should have worked	c99 would have worked	Program
<pre>{// containing block for(auto i : reverse(foo())) { std::cout << i << std::endl; } }</pre>	<pre>{// containing block auto&& rg = reverse(foo()); auto pos = rg.begin(); auto end = rg.end(); for (; pos != end; ++pos) { int i = *pos; ... } }</pre>	<pre>{// a a f { } }</pre>

It should be no different had the programmer broken a compound statement into it's components and named them individually.

CWG1864 List-initialization of array objects

```
auto x[] = {1, 2, 3};
```

I am not exactly sure what is the problem here. With `implicit constant initialization` and if `x` was `const auto` instead of `auto`, this example would not dangle. With `c99 literal enclosing block lifetime`, this example, AS IS, would not dangle.

CWG2111 Array temporaries in reference binding

"Somewhat related to P2174 compound literals."

That statement and the fact that all of the examples were `const` constant expressions means that issue is related to this proposal and the `NAD` may need to be revisited if this proposal ever gets accepted.

CWG914 Value-initialization of array types

I don't believe this issue is related to or would be benefited from this proposal.

Other Anonymous Things

The pain of immediate dangling associated with temporaries are especially felt when working with other anonymous language features of C++ such as lambda functions and coroutines.

Lambda functions

Whenever a lambda function captures a reference to a temporary it immediately dangles before an opportunity is given to call it, unless it is an immediately invoked lambda/function expression.

```
[&c1 = "hello"s](const std::string& s)// OK
{
    return c1 + " "s + s;
}("world"s);// immediately invoked lambda/function expression

auto lambda = [&c1 = "hello"s](const std::string& s)// immediate dangling
{
    return c1 + " "s + s;
}
// ...
lambda("world"s);
```

This problem is resolved when the scope of temporaries is to the enclosing block instead of the containing expression. This is the same had the temporary been named.

```
auto anonymous = "hello"s;
auto lambda = [&c1 = anonymous](const std::string& s)
{
    return c1 + " "s + s;
}
// ...
lambda("world"s);
```

This specific immediately dangling example is also fixed by implicit constant initialization provided `c1` resolves to a `const std::string&` since `c1` was constant-initialized.

Coroutines

Similarly, whenever a coroutine gets constructed with a reference to a temporary it immediately dangles before an opportunity is given for it to be `co_await`ed upon.

```

generator<char> each_char(const std::string& s) {
    for (char ch : s) {
        co_yield ch;
    }
}

int main() {
    for (char ch : each_char("hello world")) { // immediate dangling
        std::print(ch);
    }
}

```

This problem is also resolved when the scope of temporaries is to the enclosing block instead of the containing expression. This also is the same had the temporary been named.

```

generator<char> each_char(const std::string& s) {
    for (char ch : s) {
        co_yield ch;
    }
}

int main() {
    auto s = "hello world"s;
    for (char ch : each_char(s)) {
        std::print(ch);
    }
}

```

This specific immediately dangling example also is also fixed by implicit constant initialization since the parameter `s` expects a `const std::string&` and it was constant-initialized.

Unless resolved, this problem will continue to be a problem for future `C++` features, type erased or not, anonymous or not, where there are a separate construction and execution steps.

Other languages

many native languages

Many native languages, including C and C++, already provide this capability by storing said instances in the COFF String Table of the portable executable format ^[13].

Java

Java automatically perform interning on its `String` class ^[14].

C# and other .NET languages

The .NET languages also performs interning on its `String` class ^[15]

many other languages

According to Wikipedia's article, `String interning` ^[16], Python, PHP, Lua, Ruby, Julia, Lisp, Scheme, Smalltalk and Objective-C's each has this capability in one fashion or another.

What is proposed here is increasing the interning that C++ already does, but is not limited to just strings, in order to reduce dangling references. The fact is, literals in C++ is needless complicated with clearly unnecessary memory safety issues that impedes programmers coming to C++ from other languages, even C.

Summary

There are three principles repeated throughout this proposal.

1. Let constants be constants / free your constants / **implicit constant initialization**
2. **Temporaries are just anonymously named variables** / C99 compound literals lifetime rule
 - *[optional]* **general lifetime extension**: sometimes, especially in blocks / not arguments, it is better for a temporary to have automatic storage duration associated with the enclosing block of the variable to which the temporary is assigned instead of being associated with the enclosing block of the temporary expression

The advantages to C++ with adopting this proposal is manifold.

- Eliminate dangling of what should be constants
- Eliminate immediate dangling
- Reduce all remaining dangling
- Simplify C++ temporary lifetime extension rules
- Make `constexpr` literals less surprising for new and old developers alike
- Reduce the gap between C++ and C99 compound literals
- Improve the potential contribution of C++'s `constexpr` back to C
- Make string literals and C++ literals more consistent with one another
- Reduce uninitialized and delayed initialization errors
- Taking a step closer to reducing undefined behavior in string literals
- Increase and improve upon the utilization of ROM and the benefits that entails

Frequently Asked Questions

What about locality of reference?

It is true that globals can be slower than locals because they are farther in memory from the code that uses them. So let me clarify, when I say `static storage duration`, I really mean **logically** `static storage duration`. If a type is a `PODType` / `TrivialType` or `LiteralType` then there is nothing preventing the compiler from copying the global to a local that is closer to the executing code. Rather, the compiler **must** ensure that the instance is always available; **effectively** `static storage duration`.

Consider this from an processor and assembly/machine language standpoint. A processor usually has instructions that works with memory. Whether that memory is ROM or is logically so because it is never written to by a program, then we have constants.

```
mov <register>,<memory>
```

A processor may also have specialized versions of common instructions where a constant value is taken as part of the instruction itself. This too is a constant. However, this constant is guaranteed closer to the code because it is physically a part of it.

```
mov <register>,<constant>
mov <memory>,<constant>
```

What is more interesting is these two examples of constants have different value categories since the ROM version is addressable and the instruction only version, clearly, is not. It should also be noted that the later unnamed/unaddressable version physically can't dangle.

Won't this break a lot of existing code?

NO, if any. To the contrary, code that is broken is now fixed. Code that would be invalid is now valid, makes sense and can be rationally explained. Let me summarize based on the three features of this proposal.

Let constants be constants / free your constants / implicit constant initialization

This feature not only changes the point of destruction but also the point of construction. Instances that were of automatic storage duration, are now of static storage duration. Instances that were temporaries, are no longer temporaries. Surely, something must be broken! From the

earlier section “Present”, subsection “C Standard Compound Literals”. Even the `c++` standard recognized that there are other opportunities for constant initialization.

Working Draft, Standard for Programming Language C++ [2:12]

“6.9.3.2 Static initialization [basic.start.static]”

“3 An implementation is permitted to perform the initialization of a variable with static or thread storage duration as a static initialization even if such initialization is not required to be done statically ...”

So, what is the point? For the instances that would benefit from implicit constant initialization, there are currently NO guarantees as far as their lifetime and as such is indeterminate. With this portion of the proposal, a guarantee is given and as such that which was non-determinate becomes determinate.

It should also be noted that while this enhancement is applied implicitly, programmers have opted into this up to three times.

1. The programmer of the type must have provided a means for the type to be constructed at compile time likely by having a `constexpr` constructor.
2. The programmer of the variable or function parameter must have stated that they want a `const`.
3. The end programmer has `const-initialized` the variable or argument.

Having expressed constant requirements three times, it is pretty certain that the end programmer wanted a constant, even if it is anonymous.

Temporaries are just anonymously named variables / C99 compound literals lifetime rule

When programmers use temporaries, unnamed variables, instead of named variables, then they give up control of the initialization order.

“7.6.1.3 Function call [expr.call]”

*“8 The postfix-expression is sequenced before each expression in the expression-list and any default argument. The initialization of a parameter, including every associated value computation and side effect, is **indeterminately** sequenced with respect to that of any other parameter.”*

“[Note 8: All side effects of argument evaluations are sequenced before the function is entered (see 6.9.1). — end note]”

Consequently, this indeterminiteness remains regards of whether the temporary was scoped to the statement or the block. In this proposal, while the point of creation remains the same, the point of deletion gets extended just enough to remove **immediate** dangling. Since the temporary variable is by definition unnamed, any chance of breakage is greatly minimized because other than the parameter it was directly passed to, nothing else has a reference to it.

general lifetime extension

The proposed lifetimes matches or exceeds those of the current temporary lifetime extension rules in `c++`. In all of the standard examples the temporary lifetime is extended to the assigned variable.

6.7.7 Temporary objects [2:14]

```

template<typename T> using id = T;

int i = 1;
int&& a = id<int[3]>{1, 2, 3}[i]; // temporary array has same lifetime as a
const int& b = static_cast<const int&>(0); // temporary int has same lifetime as b
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
// exactly one of the two temporaries is lifetime-extended

// ...

const int& x = (const int&)1; // temporary for value 1 has same lifetime as x

// ...

struct S {
    const int& m;
};
const S& s = S{1}; // both S and int temporaries have lifetime of s

```

However, in reality, since all of these initial assignments are to the block then the real lifetime is to the block that contains the expression that contains the temporary. This is the same as the “Temporaries are just anonymously named variables” feature. However, when the initialization statement is passed directly as an argument of a function call than it is the current statement lifetime since the argument is the variable. In other words, no additional life was given. The “Temporaries are just anonymously named variables” feature improves the consistency by giving these argument temporaries the same block lifetime in order to eliminate immediate dangling and reduce further dangling. The “general lifetime extension” feature examines the `?:` ternary operator example and asks the question shouldn't this work for variables that are delayed initialized or reinitialized inside of the `if` and `else` clauses of `if/else` statements? Shouldn't the block of the variable declared above a `if/else` statement be used over the blocks of `if` and `else` clause where the temporary was assigned? This seems reasonable and would fix even more dangling. With all three features, most, if not all, moderate dangling is fixed within any given function and all we are left with are the ultra complicated and rarely done dangling or the ultra easy returning reference dangling that only occurs in predictable places between functions when returning or coawaiting.

Who would even use these features? Their isn't sufficient use to justify these changes.

Everyone ... Quite a bit, actually

Consider all the examples littered throughout this paper, these are what gets fixed

- dangling reported on normal use of the STL
- dangling examples reported in the C++ standard
- real world dangling reported in NAD, not a defect, reports

This doesn't even include the countless examples found in numerous articles comparing C++ with other nameless programming languages which would be fixed. However, the best proof can be found in our usage and other proposals.

C++ Core Guidelines [17]

F.16: For “in” parameters, pass cheaply-copied types by value and others by reference to const

In C++ , we use `const` parameters alot. This is the first of three requirements of `implicit constant initialization` . What about the use of types that can be constructed at compile time?

- **C++20** : `std::pair` , `std::tuple` , `std::string` , `std::vector`
- **C++23** : `std::optional` , `std::variant` , `std::unique_ptr`

As their was sufficient use to justify making the constructors of any one of these listed above types to be `constexpr` than their would be sufficient use of the `implicit constant initialization` feature which would use them all as this satisfies its second and third requirement that the instances be constructable at compile time and `constant-initialized` .

As far as the “temporaries are just anonymously named variables” feature, one of biggest gripes with the `Bind Returned/Initialized Objects to the Lifetime of Parameters` [1:24] proposal was that it would require a viral attribution effort. While that may be inevitable in order to fix or identify dangling in the language, that viral effort helps to identify the magnitude of C++ STL functions that has the potential of dangling in the first place and with this feature would no longer immediately dangle which is the most shocking type of dangling to end programmers.

Why not just use a static analyzer?

Typically a static analyzer doesn't fix code. Instead, it just produces warnings and errors. It is the programmer's responsibility to fix code by deciding whether the incident was a false positive or not and making the corresponding code changes. This proposal does fix some dangling but others go unresolved and unidentified. As such this proposal and static analyzers are complimentary. Combined this proposal can fix some dangling and a static analyzer could be used to identify what is remaining. As such those who still ask, “why not just use a static analyzer”, might really be saying **this proposal's language enhancements might break their**

static analyzer. To which I say, the standard dictates the analyzer, not the other way around. That is true for all tools. However, let's explore the potential impact of this proposal on static analyzers.

The `c++` language is complex. It stands to reason that our tools would have some degree of complexity, since they would need to take some subset of our language's rules into consideration. In any proposal, mine included, fixes to any dangling would result in potential dangling incidents becoming false positives between those identified by a static analyzer that overlap with said proposal. The false positives would join those that a static analyzer already has for not factoring existing language rules into consideration just as it would for any new language rules.

With `implicit constant initialization`, existing static analyzers would need to be enhanced to track the `const` ness of variables and parameters, whether or not the types of variables and parameters can be constructed at compile time and whether or not instances were constant-initialized. Until that happens, an existing dangling incident reported by static analyzer will just be a false positive. The total number of incidents remain the same and the programmer just need to recognize that it was a false positive which should be easy to do since constants are trivial and these rules are simple.

What about the `temporaries are just anonymously named variables` feature! Static analyzers need to understand the lifetimes of variables with automatic storage duration, regardless. Not quantifying the current life of any given instance and determining whether it even needs to be extended would result in false positives. This already requires tracking braces/blocks/scopes. As such, tracking the statement that contains a temporary is not significantly more complicated than tracking the block that contains said expression and temporary. In all likelihood, that is already being performed. Further, the proposed rules are significantly simpler than the current rules, once `general lifetime extension` gets factored in. This was identified by the numerous removals in the "Proposed Wording" section.

Why not just allow `constinit` to be used on function arguments?

```
some_function_call(constinit "Hello World"s);
```

I am not opposed to this, as this would provide access to much of the same benefits as `implicit constant initialization`. However, there is at least one pro and couple of con(s) to be considered. The case could be made that both be supported.

Pro

1. asserts that the argument was constant initialized

Con(s)

1. liberally/virally applied to arguments making code more verbose
2. This doesn't actually remove any dangling from the language. It would still be a manual programmer effort.

By itself, `constexpr` arguments does not address temporaries are just anonymously named variables and general lifetime extension .

Can this even be implemented?

Again, let's look at this on a feature by feature basis.

Let constants be constants / free your constants / implicit constant initialization

C++ already provides static storage duration guarantee for instances of one type and allows it for many others.

- native string literals already have static storage duration
- compilers have been free for a long time to promote compile time constructed instances to have static storage duration
- any `LiteralType` instances that are constant-initialized are already prime candidates for compilers to promote to having static storage duration

Temporaries are just anonymously named variables / C99 compound literals lifetime rule

C++ is already doing this for variable and for some instances of temporaries. What is proposed is that all temporaries should consistently benefit from this feature and even for temporaries to be made consistent with variables.

general lifetime extension

Let's say for a moment that this feature can't be implemented (easily). That is OK. It is an optional addendum to the "Temporaries are just anonymously named variables" feature. The two primary features already provides significant value whether individually or combined. It may even be advisable to release the primary functionality in one release, let the dust settle and then consider the addendum in another release.

However, this feature is simple. A compiler already knows when any variable is destroyed. So for this feature to work the compiler just needs a index on a per function basis to allow looking up the point of variable destruction given the variable again assuming this isn't already a part of

the variable metadata in the compiler. Best case this is a member access. Worst case it is a map.

This cost, additional or not, pails in comparison to proposals that fix dangling generally. Those have quadratic or exponential costs resolving variable dependency graphs. So it is a little hard to object to this proposal's cost without swearing off fixing dangling in the language altogether. Further, C++ is already doing something similar with the `?:` ternary temporary lifetime extension example; 6.7.7 *Temporary objects* [2:15].

Doesn't the `implicit constant initialization` feature make it harder for programmers to identify dangling and thus harder to teach?

If there was no dangling then there would be nothing to teach with respect to any dangling feature. Even the whole standard is not taught. So the more dangling we fix in the language, the less dangling that has to be taught to beginners. Consider the following example, does the new features make it easier or harder to identify dangling?

```
f({1,2}); // implicit constant initialization
// vs.
int i = 1;
f({i, 2}); // no implicit constant initialization
```

The `implicit constant initialization` feature is complimented by the `temporaries are just anonymously named variables` and `general lifetime extension` features. The `temporaries are just anonymously named variables` feature fixes immediate dangling whether it was `const` or not, whether it was `const-initialized` or not. It also reduces further dangling by providing life to temporary arguments after the call is made to them. It doesn't matter if the temporary argument was `implicit constant initialization` or not, as it is usually fixed, either way. As such, programmers look at temporary arguments for dangling a whole lot less. The only time it becomes an issue if the reference provided to the function is returned and propagated **manually** outside the containing scope. The `general lifetime extension` feature takes this a step further by removing or reducing dangling caused by uninitialized, delayed initialization and re-initialization.

Let's say the C++ community don't want the `temporaries are just anonymously named variables` feature, does `implicit constant initialization`, by itself, make it harder for programmers to identify dangling? In reality, it is a tradeoff, leaning heavily to easier.

It is plain to see that `{1,2}` is `constant-initialized` as it is composed entirely of `LiteralType(s)`. It is also plain to see that `{i,2}` is `modifiable` as its initialization statement is `variable` and `dynamic` due to the variable `i`. So the real questions are as follows:

- Is the first parameter to the function `f` `const`?
- Is the type of the first parameter to the function `f` a `LiteralType` ?

The fact is some programmer had to have known the answer to both questions in order to have writtern `f({1, 2})` in the first place. The case could be made that it would be nice to be able to use the `constinit` keyword on temporary arguments, `f(constinit {1, 2})` , as this would allow those who don't write the code, such as code reviewers, to quickly validate the code. Even the programmer would benefit, some, if the code was copied. However, `constinit` would mostly be superfluous, if the `temporaries` are just anonymously named variables feature is added. As such, `constinit` should be optional. Consequently, the negative impact upon identifying and teaching dangling is negligible.

Yet, the `implicit constant initialization` feature, by itself, makes it easier to identify and teach dangling.

C++ Core Guidelines

F.43: Never (directly or indirectly) return a pointer or a reference to a local object ^[18]

...

Note *This applies only to non-static local variables. All static variables are (as their name indicates) statically allocated, so that pointers to them cannot dangle.* ^[18:1]

Instances that have static storage duration can't dangle. Currently in `c++` , instances that don't immediately dangle can still dangle later such as by returning. Using `static storage duration` short circuits the dangling identification process. An instance, once identified, doesn't need to be factored into any additional dangling decision making process. Using more `static storage duration` speeds up the dangling identification process. This would also be of benefit to static analyzers that goes through a similar thought process.

Doesn't this make C++ harder to teach?

Until the day that all dangling gets fixed, any incremental fixes to dangling still would require programmers to be able to identify any remaining dangling and know how to fix it specific to the given scenario, as there are multiple solutions. Since dangling occurs even for things as simple as constants and immediate dangling is so naturally easy to produce than dangling resolution still have to be taught, even to beginners. As this proposal fixes these types of dangling, it makes teaching `c++` easier because it makes `c++` easier.

So, what do we teach now and what bearing does these teachings, the `c++` standard and this proposal have on one another.

C++ Core Guidelines

F.42: Return a τ^* to indicate a position (only) ^[19]

Note Do not return a pointer to something that is not in the caller's scope; see F.43. ^[18:2]

Returning references to something in the caller's scope is only natural. It is a part of our reference delegating programming model. A function when given a reference does not know how the instance was created and it doesn't care as long as it is good for the life of the function call (and beyond). Unfortunately, scoping temporary arguments to the statement instead of the containing block doesn't just create immediate dangling but it provides to functions references to instances that are near death. These instances are almost dead on arrival. Having the ability to return a reference to a caller's instance or a sub-instance thereof assumes, correctly, that reference from the caller's scope would still be alive after this function call. The fact that temporary rules shortened the life to the statement is at odds with what we teach. This proposal restores to temporaries the lifetime of anonymously named variables which is not only natural but also consistent with what programmers already know. It is also in line with what we teach as was codified in the C++ Core Guidelines.

Other types of dangling can still occur. One simple type is directly called out in the C++ Core Guidelines.

C++ Core Guidelines

F.43: Never (directly or indirectly) return a pointer or a reference to a local object ^[18:3]

Reason To avoid the crashes and data corruption that can result from the use of such a dangling pointer. ^[18:4]

Other than turning some of these locals into globals, this proposal does not solve nor contradict this teaching. If anything, by cleaning up the other dangling it makes the remaining more visible. Also by hollowing out the majority and most common dangling in the middle, programmers are left with only these ultra trivial, such as F.43 ^[18:5], or the ultra rare and ultimately more complex dangling, which is naturally avoided by keeping one's code simple.

Further, what is proposed is easy to teach because we already teach it and it makes c++ even easier to teach.

- We already teach that native string literals don't dangle because they have static storage duration. This proposal just extends the concept to constants, as expected. This increases good consistency and reduces a bifurcation that is currently taught.
- We already teach RAII and that local variables are scoped to the block that contains them. This proposal just extends the concept to temporaries. This increases good consistency and removes or reduces multiple bifurcations that are currently taught;
 - that certain temporaries gets extended when assigned to a block but not when assigned to a argument

- that variables and temporaries are all that different
- that named and unnamed variables are different

All of this can be done without adding any new keywords or any new attributes. We just use constant and variable concepts that beginners are already familiar with.

c++ programmers are trapeze artists flying over the flames of dangling. This proposal shines a spotlight on the safety net that c++ already has via constants on one axis and variables on the opposing axis. Isn't it high time we raise the net off of the ground floor so it can help catch us when we inevitably fall.

References

- [illegible]

7. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1511.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1511.pdf>) ↔
8. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2235.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2235.pdf>) ↔
9. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2731.pdf> (<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2731.pdf>) ↔
10. <https://gcc.gnu.org/onlinedocs/gcc/Compound-Literals.html> (<https://gcc.gnu.org/onlinedocs/gcc/Compound-Literals.html>) ↔
11. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1018r16.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1018r16.html>) ↔
12. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r1.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r1.pdf>) ↔
13. <https://docs.microsoft.com/en-us/windows/win32/debug/pe-format> (<https://docs.microsoft.com/en-us/windows/win32/debug/pe-format>) ↔
14. [https://docs.oracle.com/javase/7/docs/api/java/lang/String.html#intern\(\)](https://docs.oracle.com/javase/7/docs/api/java/lang/String.html#intern()) (<https://docs.oracle.com/javase/7/docs/api/java/lang/String.html#intern%28%29>) ↔
15. <https://docs.microsoft.com/en-us/dotnet/api/system.string.intern?view=net-6.0> (<https://docs.microsoft.com/en-us/dotnet/api/system.string.intern?view=net-6.0>) ↔
16. https://en.wikipedia.org/wiki/String_interning (https://en.wikipedia.org/wiki/String_interning) ↔
17. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-in> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-in>) ↔

18. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object>) ↩ ↩ ↩ ↩ ↩ ↩
19. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only>) ↩